ConnectFS: A Real-time Internet Data Aggregator File System

Zani Xu
Harvard University

Abstract

ConnectFS is a real-time internet data aggregator filesystem that extends the Unix "everything is a file" abstraction to the internet. Unix traditionally represents hardware devices, process metadata, and kernel parameters as local files, but ConnectFS enables users to access internet data using standard command-line tools such as cat and grep. The system utilizes a three-tier architecture in order to bridge the latency gap between local access and remote APIs. Tier 1 utilizes a FUSE-based daemon to intercept system calls and displays the data as a local virtual filesystem. Tier 2 consists of a Flask aggregator server that makes API requests and gives data to Tier 1 while utilizing polling and caching to maintain freshness while still keeping local latency low. Tier 3 consists of the external providers, which for the sake of this project is Yahoo Finance. The current implementation of the project supports dynamic updates through a writable .config file, allowing the user to change which tickers are being cached and polled.

1 Introduction

The development of ConnectFS was prompted by a desire to explore Linux storage systems. The cornerstone of Unix design is the abstraction that "everything is a file" [3]. By representing hardware devices (/dev), process metadata (/proc), and kernel parameters (/sys) as files, Unix allows users to interact with complex systems using standard tools like cat, grep, and awk or shell scripts. Through this unified interface, the same set of commands can be used to manage a physical hard drive or monitor a running process. However, currently, this elegance is largely confined to local resources. While local data is easily accessible, the vast wealth of information on the Internet is not. If a quantitative research team or developer wants to use live data, they are often forced to write complex scripts to handle various RESTful APIs and manage diverse JSON objects or dataframes. This creates a significant barrier to entry, as these developers have to spend more time on data engineering rather than analysis. My project, ConnectFS, aims to solve this by extending the Unix abstraction to the Internet. By aggregating heterogeneous real-time Internet data as a local virtual filesystem, ConnectFS enables users to access live Internet data with the same simplicity as local text files. By presenting Internet data as a standard directory of files, researchers can execute models against local paths, abstracting away the complexities of web requests and data sourcing. Furthermore, through a three-tier architecture, ConnectFS is able to bridge the latency gap between high-speed local filesystem access and the relatively slow and unpredictable connections of remote data providers. ConnectFS transforms the Internet from a collection of APIs into a seamless extension of the local operating system.

At first, ConnectFS was meant to be used for a variety of types of data. However, I later refined the scope of the project to focus on financial data, such as real-time stock tickers. The core architecture is designed to be flexible, allowing it to be easily adapted for aggregating any web service data into a Linux filesystem.

2 Design and Architecture

The design of ConnectFS is centered around a 3-tier architecture. By separating the system into distinct layers, ConnectFS can maintain the responsive feel of a local disk while managing the complex and slow network operations in the background. The system aggregates data through the following tiers:

Tier 1: Client-Side ConnectFS Daemon Process with Filesystem in Userspace (FUSE) This layer creates the virtual mount point using FUSE. It intercepts standard system calls and translates these local file paths into network requests directed at the aggregator server.

Tier 2: ConnectFS Data Aggregator Server This layer acts as the middleman of the system, and is powered by Flask.

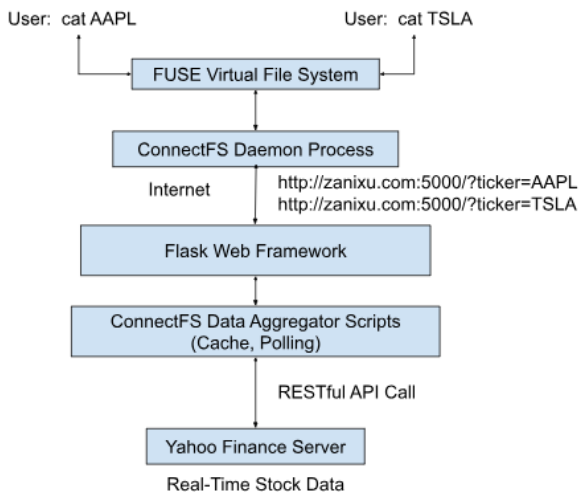

Figure 1: ConnectFS 3-Tier System Architecture: Showing the flow from local FUSE system calls to the Flask-based aggregator and Tier 3 data providers.

It handles incoming JSON objects from external sources, and manages caching and polling mechanisms to ensure data freshness without overloading external providers.

Tier 3: Various External Data Providers This tier is where the original sources of information lie. For my project, the system connects to the Yahoo Finance server to pull real-time market data and financial statements.

A diagram of the architecture of ConnectFS and the flow of the data is shown in Figure 1.

2.1 Tier 2: ConnectFS Data Aggregator Server

The ConnectFS Data Aggregator Server serves as the middleman of the architecture. Its main purpose is to give the Tier 1 local filesystem fast access to the data while also keeping the data as fresh as possible from the Tier 3 data providers. The data aggregator needs to be able to handle data as dataframes or JSON objects through RESTful APIs, so I chose to power the server through the Flask web framework, as it is efficient in handling diverse data structures [2]. After setting up a domain name (zanixu.com), the Flask server displays the incoming JSON objects received through the Yahoo Finance APIs, with the ticker being passed in through the HTTPS query parameters. In my first implementation, this data was fetched on-request, with the Flask server sending an API request whenever data was requested from the site. However, this did not allow for a high-speed local filesystem as accessing a file would require sending an API request, which is relatively slow. In order to maintain the "everything is a file" elegance without sacrificing performance, Tier 2 implements some critical optimization mechanisms, utilizing the following:

Caching: The server manages an internal cache to serve Tier 1 requests for data instantly, reducing redundant network round-trips to Tier 3, the external providers.

Polling: To keep data fresh while avoiding "overloading" these external providers, the server utilizes background polling to pre-fetch updates for active data streams.

I also implemented a writable control file (.config). This contains the list of tickers for Tier 2 to cache and poll, and allows users to dynamically update the available files in the directory without unmounting. Because of this, when Tier 1 makes a request to Tier 2 for some data, a fresh version of it will already be cached on the server and can be sent back immediately.

2.2 Tier 1: Client-Side ConnectFS Daemon Process with FUSE

The Tier 1 Client-Side ConnectFS daemon process with FUSE provides the user-facing interface that achieves the "everything is a file" philosophy. This layer creates a bridge between the standard operating system calls and the remote data aggregation in Tier 2.

FUSE Virtual Filesystem:

At the core of Tier 1 is Filesystem in Userspace (FUSE), which allows for the creation of a virtual mount point without needing to modify the Linux kernel. I implemented a mountable FUSE application using the libfuse library. When a user executes a standard command, such as cat or ls, the FUSE system intercepts the system call at the kernel level.

Daemon Process:

These intercepted calls are passed to the ConnectFS daemon process, which is a background process that manages the logic for the virtual filesystem. This process translates the virtual file paths into specific network requests directed at the Tier 2 aggregator server.

This tier also supports dynamic configuration though the writable control file; by writing a list of stock tickers to this file, the user can make real-time adjustments to the content of the filesystem without the need to unmount or restart the daemon process.

3 Overview and Future Work

Due to the successful virtual filesystem implementation as well as the development of the writable control file, my project hit the 100% goal. However, the performance of ConnectFS

was still not as high as I hoped. Due to dynamic throttling and IP-based blocking from the Yahoo Finance API, designed to mitigate against automated scraping, the frequency of external requests needed to be reduced in order to keep the server functional. While ConnectFS is highly effective for immutable historical data, this means that for real-time market data, while it is still being polled and updated frequently, may not necessarily be the most fresh it could be. Nonetheless, ConnectFS successfully demonstrates the viability of extending the classic "everything is a file" philosophy to the Internet. By implementing a 3-tier architecture, the project bridges the performance gap between high-speed local filesystem operations and the latency of remote API providers. The system achieves its core objective of abstracting the complexities of RESTful APIs into a simple and scriptable directory of text files. ConnectFS provides a unified interface for real-time financial data sourcing, with a flexible architecture that can be easily adapted for aggregating other data types.

While the current implementation provides a stable foundation, there are still several ways to enhance the system's efficiency and scale. Although not important for this specific project, when larger datasets are involved, there may be cases where only specific parts of files need to be accessed. Future iterations could leverage HTTP Range [1] headers to allow the daemon to "seek" through remote data and request specific byte ranges, bypassing the need to download entire files. Furthermore, further optimization of the background polling scripts is required to improve the freshness of volatile market data.

4 AI Usage Report

4.1 How I used AI

I used AI in a variety of different ways throughout this project, both to help me gain a better understanding of the topics in my project as well as to assist in the development of the project itself.

Because I wanted to do a final project centered on Linux, something I had very little experience with, I utilized Google Gemini to teach me the basics as well as how storage works in Linux systems; this is how I learned about the "everything is a file" abstraction, which I found interesting and based my project off of. Furthermore, I used Google Gemini to help write some code for my project, making sure that the results met my expectations and conversing with it to get what I wanted. Due to my general unfamiliarity with Linux, I also used AI to help debug any issues that would come up and explain what was going wrong to give me a better understanding of Linux.

4.2 What I Did Not Expect

Obviously I knew that AI would not give me exactly what I wanted without constant tweaking and revisions; however, I was surprised that sometimes it straight up didn't understand the correct formatting for some libraries and functions. I expected that all the bugs created by code generated from AI would be due to the AI writing code that differed from my intent, but sometimes it would do things like swap parameters in functions or remove necessary functions causing the programs to not even compile.

4.3 What I Learned and Useful Tips

Through this project, I learned that giving AI broad queries was unlikely to give me what I wanted. Instead, figuring out exactly how I wanted to design my programs and then providing the AI with specific technical constraints was much more effective and resulted in higher-quality code. I also learned that AI was very good at parsing through errors; I often find that I don't really understand what the multi-line long error codes are saying, but AI is able to explain them to me. One last overarching tip I would give for when someone is using AI to add more features to some code is to make sure to emphasize to add just what is necessary in order to make the change and keep the rest the same, as a lot of the times it would try to rewrite a lot of the code and randomly remove another important feature, defeating the purpose.

References

[1] R. T. Fielding, Y. Lafon, and J. Reschke. Hypertext transfer protocol (http/1.1): Range requests. RFC 7233, RFC Editor, 2014. https://datatracker.ietf.org/doc/html/rfc7233.

[2] Miguel Grinberg. *Flask Web Development*. O'Reilly Media, Incorporated, 2018.

[3] Linus Torvalds. Everything is a file descriptor or a process. Yarchive. https://yarchive.net/comp/linux/everything_is_file.html.